

```
MONGO_URI=mongodb+srv://priyanshsahu157:siGOVsuCCz1ebkYY@cluster0.t9m7wwg.mongodb.net/spotify_notification
JWT_SECRET=ae04bd911f40c69a82aeccfef62f80d46af78efcac3e997b

CLIENT_ID=742989631912-u64g1od47h0kgod8p2q660h6slavicfd.apps.googleusercontent.com
CLIENT_SECRET=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
#
REFRESH_TOKEN=1//04cAS7EGv1JheCgYIARAAGASNWf-L9IrA6f7qQnMhXH
aR1__9UCEyPV9nq-7UGQ7VRzRqav7tXE1YNeSXaBm_GGy2zPm4TfabhA
REFRESH_TOKEN=1//04tQkZQUwo-NJCgYIARAAGASNWf-L9Ir9mXmu9T3lXo
cuRnDfkFPGyKTVGVGb1ZK2jB5dgUBoLRNUyTxDcqqwxAE4A6OKkEMEF4
EMAIL_USER=priyanshsahu157@gmail.com

RABBITMQ_URL=amqp://nqpwtgbw:fhSU-Ry09kRgC0dtZ9v-193q_RrB2QH
f@fuji.lmq.cloudamqp.com/nqpwtgbw
```

Hello sir/ma'am, my name is Priyansh Sahu, and I am currently pursuing B.Tech in Computer Science from Prestige Institute, Bhopal.

I have strong fundamentals in Data Structures and Algorithms using C++, along with a good understanding of Object-Oriented Programming concepts.

I have also worked on full-stack projects using the MERN stack, including an AI-powered chat application and a real-time messaging platform, which helped me strengthen my problem-solving and development skills.

I am a quick learner and highly interested in software development roles where I can apply my skills and contribute effectively to the organization.

PROJECT BASED (MOST IMPORTANT)

1. Explain your project (BEST ANSWER)

 Use this format:

Answer:

I worked on a project called *Nexus-Talk*, which is a real-time messaging platform built using the MERN stack. It allows users to communicate instantly using Socket.IO. I implemented JWT-based authentication for secure login and used MongoDB for storing messages.

This project helped me understand real-time communication, API design, and handling multiple users efficiently.

2. How real-time chat works?

Answer:

Real-time chat works using persistent connections between client and server. Instead of repeated requests, the server pushes messages instantly using technologies like WebSockets or Socket.IO, enabling low-latency communication.

3. What is WebSocket / Socket.IO?

Answer:

WebSocket is a protocol that provides full-duplex communication between client and server over a single connection.

Socket.IO is a library built on top of WebSocket that simplifies real-time communication by handling reconnections, events, and cross-browser compatibility.

4. How you handled authentication (JWT)?

Answer:

I used JWT (JSON Web Token) for authentication. When a user logs in, the server generates a token, which is sent to the client. The client stores it and sends it with every request. The server verifies the token to allow secure access.

✓ 5. How you improved performance (Redis)?

Answer:

I used Redis for caching frequently accessed data, which reduced database queries and improved response time. This helped in making the application faster and more scalable.

✓ 6. What challenges you faced?

Answer:

One challenge was handling real-time synchronization between users. I solved it using Socket.IO events. Another challenge was optimizing performance, which I improved using caching and efficient API design.

 BACKEND (Node.js / Express)

✓ 7. What is middleware?

Answer:

Middleware is a function in Express that runs between request and response. It can modify request/response, handle authentication, or perform logging.

✓ 8. What is next()?

Answer:

`next()` is a function used to pass control to the next middleware in the pipeline.

✅ 9. How API request flow works?

Answer:

Client sends request → Middleware processes it → Route handler executes logic → Response is sent back to client.

✅ 10. How you handle errors?

Answer:

I use centralized error-handling middleware in Express to catch errors and send proper responses with status codes.

🏆 DATABASE (MongoDB + Redis)

✅ 11. What is indexing?

Answer:

Indexing improves query performance by allowing faster data retrieval instead of scanning the entire collection.

✅ 12. Difference: find vs aggregate

Answer:

- `find()` → simple queries
- `aggregate()` → complex operations like grouping, filtering, and transformations

✓ 13. MongoDB basics

Answer:

MongoDB is a NoSQL database that stores data in JSON-like documents. It is flexible and scalable.

✓ 14. How you optimized queries?

Answer:

I used indexing, avoided unnecessary data fetching, and used caching to reduce database load.

✓ 15. What is caching (Redis)?

Answer:

Caching stores frequently accessed data in memory, so it can be retrieved faster instead of querying the database every time.

✓ 16. Redis purpose

Answer:

Redis is an in-memory database used for caching, session management, and improving application performance.

“A RESTful API allows client and server to communicate using HTTP requests like GET, POST, PUT, DELETE in a structured way.”

Git is a distributed version control system that helps track changes, manage code history, and collaborate with teams.”

3. What is merge conflict?

👉 Answer:

“It happens when two branches modify the same part of code. Git cannot decide automatically, so we resolve it manually.”

JavaScript Interview (Must Know)

1. Data Types

👉 Primitive:

- String, Number, Boolean, null, undefined, Symbol

👉 Non-primitive:

- Object, Array

 Answer:

“JavaScript has primitive and reference data types. Primitive are immutable, while objects are mutable.”

2. let vs const vs var (VERY COMMON)

Feature	var	let	const
Scope	Function	Block	Block
Reassign	✓	✓	✗

 Best answer:

“let and const are block-scoped, while var is function-scoped. const cannot be reassigned.”

3. Closures (IMPORTANT)

 Definition:

“A closure is a function that remembers variables from its outer scope even after the outer function has executed.”

 Example:

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  };  
}
```

4. Event Loop (HIGH CHANCE)

 Answer:

“JavaScript is single-threaded but uses an event loop to handle asynchronous operations using callback queue and call stack.”

5. Promises & Async/Await

 Promise states:

- Pending
- Resolved

- Rejected

 Answer:

“Promises handle async operations, and async/await makes them easier to write and read.”

6. == vs ===

- == → loose equality
- === → strict equality

 Interview line:

“Always prefer === to avoid type coercion issues.”

React.js Interview (HIGH PRIORITY)

1. What is React?

 Use this:

“React is a JavaScript library used to build user interfaces using components.”

2. What are Components?

- Reusable UI blocks

 Types:

- Functional components (you used this)

3. What is useState?

 Answer:

“useState is a React hook used to manage state in functional components.”

4. What is useEffect?

 Answer:

“useEffect is used to handle side effects like API calls, subscriptions, or DOM updates.”

5. Virtual DOM (IMPORTANT)

 Answer:

“React uses Virtual DOM to optimize updates by comparing changes and updating only necessary parts.”

6. Props vs State

Props	State
Read-only	Mutable
Passed from parent	Managed inside component

7. Controlled Components

👉 Answer:

“Form elements controlled by React state.”

8. Why React?

👉 Answer:

“Reusable components, fast rendering with Virtual DOM, and strong ecosystem.”

Questions Based on YOUR Projects

Since you used React, expect:

♦ From Nexus-Talk / AuroraChat:

- How you handled API calls?
- How state is managed?
- How UI updates in real-time?

👉 You can say:

“I used useState and useEffect to manage state and API calls, and integrated Socket.IO for real-time updates.”

Final Strategy (VERY IMPORTANT)

👉 Don't try to learn everything

👉 Just master these 10–12 topics

100 FINAL INTERVIEW ANSWER (SAFE LINE)

“I have worked extensively with JavaScript concepts like closures, promises, and async/await. In React, I have built projects using functional components, hooks like useState and useEffect, and handled API integration and real-time updates.”

What is Node.js?

👉 Node.js is a runtime environment that allows you to run JavaScript **outside the browser**.

📌 Best answer:

“Node.js is a JavaScript runtime built on Chrome’s V8 engine that allows us to build scalable backend applications.”

Key Features of Node.js

- Non-blocking (asynchronous)
- Event-driven
- Single-threaded
- Fast execution

📌 Interview line:

“Node.js uses a non-blocking, event-driven architecture, making it efficient for handling multiple requests.”

Event Loop (VERY IMPORTANT)

👉 Core concept of Node.js

📌 Answer:

“The event loop allows Node.js to handle multiple requests asynchronously using a single thread by delegating tasks to the system and processing callbacks.”

What is Express.js?

👉 Express.js is a lightweight framework for building web applications and APIs in Node.js.

📌 Best answer:

“Express.js is a minimal web framework for Node.js used to build RESTful APIs and handle routing, middleware, and server logic.”

Why Express.js?

- Simplifies server creation
 - Routing is easy
 - Middleware support
 - Faster development
-

Basic Express Server

```
const express = require("express");  
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Hello World");  
});
```

```
app.listen(3000);
```

Important Concepts (MUST KNOW)

1. Routing

👉 Defines endpoints

```
app.get("/users", (req, res) => {})  
app.post("/login", (req, res) => {})
```

2. Middleware (VERY IMPORTANT)

👉 Functions that run before request reaches route

📌 Example:

```
app.use((req, res, next) => {  
  console.log("Middleware running");  
  next();  
});
```

📌 Interview line:

“Middleware functions process requests before they reach the route handler.”

3. next() (MOST ASKED)

📌 Answer:

“next() passes control to the next middleware function in the stack.”

4. Request & Response

- `req` → client data

- `res` → send response
-

5. Error Handling

```
app.use((err, req, res, next) => {  
  res.status(500).send("Error");  
});
```

Real Use in YOUR Project (VERY IMPORTANT)

 Say this confidently:

“I used Node.js and Express.js to build RESTful APIs for authentication, messaging, and data handling. I implemented middleware for authentication using JWT and handled API routing efficiently.”

Top Interview Questions (HIGH CHANCE)

1. Node.js vs Express.js

 Answer:

“Node.js is a runtime environment, while Express.js is a framework built on top of Node.js.”

2. What is middleware?

👉 Already covered

3. What is next()?

👉 Already covered

4. How API request flows?

👉 Answer:

“Client → Middleware → Route Handler → Controller Logic → Database → Response”

5. How do you handle errors?

👉 Answer:

“Using centralized error-handling middleware in Express.”

6. What is REST API?

👉 Already prepared 👍

7. How do you handle authentication?

👉 Answer:

“Using JWT tokens stored on client side and verified in middleware.”

700 **Final Interview Answer (Use This)**

“I use Node.js as a backend runtime and Express.js as a framework to build RESTful APIs. I have implemented routing, middleware, and authentication using JWT in my projects. I also handled real-time communication using Socket.IO and optimized performance using Redis.”